

Aula 6 — Autômatos Finitos

Compiladores I

FACOM - Faculdade de Computação

Bloco 3: Análise Léxica

Duração estimada: 70–80 minutos

Revisão da Aula 5

- ▶ Diagramas de transição reconhecem padrões léxicos;
- ▶ Alguns tokens exigem lookahead.

Pergunta rápida

O que um diagrama de transição precisa ter para reconhecer um token com certeza (estado de aceitação)?

Objetivos de aprendizagem

- ▶ Diferenciar autômatos finitos determinísticos (AFD) e não determinísticos (AFN);
- ▶ Converter uma expressão regular em um AFN (Thompson) e depois em AFD (subconjuntos);
- ▶ Aplicar o processo de minimização de um AFD.

Sumário da aula

AFD x AFN

De expressão regular a AFD

Autômato Finito Determinístico (AFD)

Definição formal

$M = (Q, \Sigma, \delta, q_0, F)$: conjunto de estados Q , alfabeto Σ , função de transição $\delta : Q \times \Sigma \rightarrow Q$ (única!), estado inicial q_0 , estados finais F .

Em um AFD, para cada estado e símbolo, existe **no máximo uma** transição possível — é exatamente o que vimos como diagrama de transição na Aula 5.

Autômato Finito Não Determinístico (AFN)

Diferenças em relação ao AFD

- ▶ Pode haver **mais de uma** transição para o mesmo símbolo em um estado;
- ▶ Pode haver transições vazias (ε), sem consumir caractere;
- ▶ Mais fácil de construir a partir de uma expressão regular, porém mais difícil de simular diretamente.

Pergunta rápida

Por que um AFN é "mais fácil de construir" a partir de uma expressão regular do que um AFD?

Sumário da aula

AFD x AFN

De expressão regular a AFD

Construção de Thompson ($ER \rightarrow AFN$)

- ▶ Regra para cada operador da expressão regular (concatenação, união, fecho de Kleene);
- ▶ Cada subexpressão vira um pequeno AFN com um único estado inicial e um único estado final, conectados por transições ε ;
- ▶ O AFN completo é a composição desses blocos.

Exemplo —

Para $a|b$: um novo estado inicial se ramifica por ε para os AFNs de a e b , que convergem por ε para um novo estado final.

Construção de subconjuntos ($AFN \rightarrow AFD$)

Ideia central

Cada estado do AFD equivalente corresponde a um **conjunto** de estados do AFN que podem estar ativos simultaneamente (fecho- ε).

- ▶ Começa pelo fecho- ε do estado inicial do AFN;
- ▶ Para cada símbolo do alfabeto, calcula o novo conjunto de estados alcançáveis;
- ▶ Repete até não haver novos conjuntos — garante determinismo.

Minimização de AFD

- ▶ Objetivo: obter o AFD equivalente com o **menor número de estados** possível;
- ▶ Ideia geral: agrupar estados "equivalentes" (que levam ao mesmo comportamento futuro) em uma única classe;
- ▶ Importante na prática: reduz o tamanho da tabela de transições usada pelo scanner gerado.

Pergunta rápida

Por que minimizar o AFD é relevante para o desempenho de um scanner real, como o do Micro C?

Desafio em sala

Desafio em sala

Considere a expressão regular $(a|b)^*abb$ (reconhece cadeias terminadas em abb). Em grupo: (1) esboce o AFN pela construção de Thompson para a subexpressão $(a|b)$; (2) discuta quantos estados um AFD equivalente precisaria ter, no mínimo.

Síntese da aula

- ▶ AFD tem transições únicas; AFN permite múltiplas transições e transições ε ;
- ▶ Toda expressão regular pode virar um AFN (Thompson) e todo AFN pode virar um AFD equivalente (subconjuntos);
- ▶ A minimização reduz o AFD ao menor número de estados equivalente.

Próxima aula: projetar um analisador léxico real, com base nesses autômatos.

Teste rápido — Aula 6

Questões teóricas

1. Qual a principal diferença entre um AFD e um AFN?
2. Para que serve a construção de subconjuntos?

Questão prática

3. Desenhe o AFD que reconhece exatamente a linguagem $\{a^n \mid n \geq 1\}$ (uma ou mais letras "a").